

Frameworks, ORMs e o mapeamento de relacionamentos

Um estudo aplicado da construção de uma API REST com Fastify, TypeScript e Drizzle ORM

Integrantes do grupo

Apresentados em ordem alfabética.

Nome completo	Matrícula
Callebe de Almeida Sousa	02410041073
Ediglei Davi Alexandre de Sousa	02410041115
Taoã Augusto Martinelli de Araújo	01410041196

Resumo

Este artigo documenta as decisões técnicas tomadas na construção de uma API REST com Fastify, TypeScript e Drizzle ORM sobre PostgreSQL. O texto parte de uma discussão conceitual sobre o que são frameworks e ORMs, quais problemas eles resolvem e quais custos impõem, e em seguida examina as particularidades de Fastify e Drizzle. A parte central trata da implementação de relacionamentos, tomando como caso de estudo a ligação entre produto e categoria: discute-se a distinção entre a chave estrangeira declarada no banco e o grafo de relações declarado na aplicação, o SQL efetivamente gerado pela Relational Query API do Drizzle, a extensão do modelo para um relacionamento muitos-para-muitos e os limites do que a integridade referencial consegue garantir sozinha.

Palavras-chave: Fastify; Drizzle ORM; TypeScript; PostgreSQL; mapeamento objeto-relacional; integridade referencial.

1 Introdução

Construir uma API que lê e escreve em um banco relacional é uma tarefa repetitiva o suficiente para que praticamente ninguém a faça do zero. Entre o socket TCP e a regra de negócio existe uma quantidade grande de trabalho genérico — interpretar o protocolo HTTP, rotear a requisição, validar a entrada, abrir conexão com o banco, traduzir linhas em objetos, serializar a resposta — e é esse trabalho que frameworks e ORMs se propõem a absorver.

A escolha dessas ferramentas, porém, não é neutra. Cada abstração que poupa código também esconde comportamento, e comportamento escondido reaparece na forma de consultas lentas, erros incompreensíveis ou restrições que só se manifestam tarde. O objetivo deste artigo é examinar esse equilíbrio com base em um caso concreto: a API desenvolvida para esta atividade, que modela um catálogo de cardápios sobre sete tabelas relacionadas.

O modelo implementado organiza estabelecimentos (*empresas*) localizados em *cidades* de uma *uf*. Cada empresa classifica seus produtos — aqui chamados *itens* — em *categorias*, e combina esses itens em *pratos*. Sempre que o texto se referir ao relacionamento entre **produto e categoria**, trata-se da ligação entre as tabelas *itens* e *categorias*; a discussão vale, sem alteração, para qualquer par de entidades com a mesma cardinalidade.

O artigo está organizado em cinco partes. As seções 2 e 3 tratam de frameworks em geral e do Fastify em particular. As seções 4 e 5 fazem o mesmo percurso para ORMs e para o Drizzle. A seção 6, mais extensa, examina a implementação dos relacionamentos, que é o ponto central da atividade.

2 Frameworks

2.1 Biblioteca e framework: a inversão de controle

A diferença entre uma biblioteca e um framework não está no tamanho nem na quantidade de funcionalidades, mas em quem chama quem. Ao usar uma biblioteca, o programador escreve o fluxo principal e invoca a biblioteca quando precisa dela. Ao usar um framework, o fluxo principal pertence ao framework, e o programador preenche os pontos de extensão que ele oferece. Esse arranjo é conhecido como **inversão de controle**, e é resumido pelo chamado Princípio de Hollywood: *não nos chame, nós chamamos você*.

A consequência prática é que um framework impõe uma estrutura. Ele define onde as rotas são declaradas, como os erros sobem, em que ordem os interceptadores rodam. Isso é simultaneamente a maior vantagem e a maior limitação: a estrutura elimina decisões repetidas e torna o código previsível para quem chega depois, mas exige que o problema caiba no formato que o framework assume.

2.2 O que se ganha e o que se paga

O ganho mais evidente é a quantidade de código que deixa de ser escrita. Um servidor HTTP escrito diretamente sobre o módulo `node:http` exige tratar manualmente o parsing do corpo da requisição, o roteamento por método e caminho, os cabeçalhos de resposta e o tratamento de exceções — problemas resolvidos, sem variação interessante, em toda aplicação.

Menos evidente, e mais valioso, é o ganho em **decisões convergentes**. Quando o framework define que a validação acontece antes do handler, todos os handlers do projeto podem assumir que os dados já chegaram válidos. Essa garantia compartilhada é o que permite que o código de negócio fique curto.

Em contrapartida, paga-se em três moedas. A primeira é a curva de aprendizado: é preciso conhecer as convenções antes de ser produtivo. A segunda é o acoplamento: trocar de framework costuma significar reescrever a camada HTTP inteira. A terceira, e mais sutil, é a **abstração vazada** — quando algo dá errado dentro do framework, é necessário entender o que ele faz por baixo, justamente o que a abstração prometia esconder.

2.3 Fastify no ecossistema Node.js

No ecossistema Node.js, o Express foi por muitos anos o padrão de fato. Sua API mínima — middlewares encadeados sobre requisição e resposta — popularizou-se justamente por impor pouco. Esse minimalismo, no entanto, transfere para a aplicação decisões que se repetem em todo projeto: como validar entrada, como padronizar erros, como documentar as rotas.

O Fastify parte de premissas diferentes. Ele trata o **schema da rota como parte da sua definição**, e não como uma preocupação externa; adota um sistema de plugins com encapsulamento explícito; e trata desempenho como requisito de projeto, não como consequência. Essas três decisões explicam a maior parte das suas particularidades, examinadas a seguir.

3 Fastify: particularidades

3.1 Arquitetura de plugins e encapsulamento

Em Fastify, quase tudo é um plugin: um conjunto de rotas, uma conexão com o banco, um interceptador de autenticação. Um plugin é apenas uma função que recebe a instância do servidor e registra coisas nela.

O ponto que distingue o Fastify é o **encapsulamento**. Quando um plugin é registrado com `app.register(...)`, ele recebe uma instância derivada, e tudo que adiciona a ela — hooks, decorators, outros plugins — fica visível apenas dentro daquele escopo e de seus descendentes. O efeito é que um plugin de autenticação registrado dentro de um grupo de rotas administrativas não vaza para as rotas públicas, sem que seja preciso repetir a verificação rota a rota nem tomar cuidado com a ordem de declaração.

Na API desenvolvida, cada recurso é um plugin independente, e o arquivo `src/routes/index.ts` apenas os compõe:

```
export const registerRoutes: FastifyPluginAsyncZod = async (app) => {
  await app.register(ufsRoutes)
  await app.register(cidadesRoutes)
  await app.register(empresasRoutes)
  await app.register(categoriasRoutes)
  await app.register(itensRoutes)
  await app.register(pratosRoutes)
}
```

Composição das rotas. Cada módulo desconhece os demais; acrescentar um recurso novo é acrescentar uma linha.

3.2 Schemas como contrato: validação e serialização

A particularidade mais consequente do Fastify é o tratamento dado a schemas. Cada rota pode declarar a forma esperada de `body`, `querystring`, `params` e de cada resposta. O framework usa essa declaração para três finalidades distintas a partir de uma única fonte.

A primeira é a **validação da entrada**, executada antes do handler. A segunda é a **geração da**

documentação OpenAPI, derivada dos mesmos schemas. A terceira, e a menos óbvia, é a **serialização da saída**: conhecendo antecipadamente o formato da resposta, o Fastify compila uma função especializada de serialização em vez de recorrer ao `JSON.stringify` genérico. Como o formato é conhecido, não há necessidade de inspecionar o objeto em tempo de execução para descobrir sua estrutura.

A serialização guiada por schema tem ainda um efeito de segurança que costuma passar despercebido: campos não declarados no schema de resposta são **removidos** da saída. Um objeto que carregue, por acidente, uma coluna sensível não a expõe, porque ela não consta do contrato.

Neste projeto, os schemas foram escritos em **Zod** e conectados ao Fastify pelo `fastify-type-provider-zod`. A vantagem sobre o JSON Schema puro é que o Zod é, ao mesmo tempo, validador em tempo de execução e fonte de tipos em tempo de compilação — o tipo de `request.body` é inferido do schema, sem declaração duplicada:

```
const criarCategoriaSchema = z.object({
  empresaId: z.uuid({ message: 'Informe o id (UUID) de uma empresa existente.' }),
  nome: z.string().trim().min(2).max(100),
})

app.post('/categorias', {
  schema: {
    body: criarCategoriaSchema,
    response: { 201: categoriaSchema, ...errorResponses },
  },
}, async ({ body }, reply) => {
  // `body` já chega validado E tipado como { empresaId: string; nome: string }
  const [categoria] = await db.insert(categorias).values(body).returning()
  return reply.status(201).send(categoria!)
})
```

Um único schema serve à validação, à tipagem e à documentação da rota.

3.3 Ciclo de vida e tratamento centralizado de erros

O Fastify expõe o processamento de uma requisição como uma sequência de etapas nomeadas, em que é possível intervir por meio de *hooks*. Duas dessas extensões foram decisivas na organização deste projeto: o `setErrorHandler` e o `setNotFoundHandler`.

Com um tratador de erros central, **nenhuma rota da aplicação precisa de try/catch**. As rotas lançam erros de domínio — `NotFoundError`, `ConflictError` — e o tratador decide o status HTTP e o formato da resposta. O resultado é que a regra de negócio não menciona códigos HTTP, e o formato de erro da API é definido em um único lugar.

O mesmo tratador converte erros vindos do PostgreSQL. Uma violação de unicidade chega como o código SQLSTATE `23505`, acompanhada do nome da constraint violada; mapear esse nome permite responder com uma mensagem compreensível em vez de expor o erro bruto do banco:

```
if (erroDoBanco.code === PG_ERROR.UNIQUE_VIOLATION) {
  return reply.status(409).send({
    message: CONSTRAINT_MESSAGES[constraint] ?? 'Ja existe um registro com esses dados.',
    code: 'UNIQUE_VIOLATION',
  })
}
```

Tradução de constraint do banco para mensagem de usuário, no tratador central.

Um detalhe descoberto apenas em teste com banco real: o Drizzle não propaga o erro do driver diretamente — ele o encapsula em um `DrizzleQueryError` e coloca o erro original em `cause`. Verificar `error.code` no nível de cima fazia toda violação de constraint cair no 500 genérico. A solução foi percorrer a cadeia de `cause` até encontrar um código com formato `SQLSTATE` (cinco caracteres alfanuméricos), o que também evita confundir o erro do banco com erros do próprio Fastify, que usam códigos no formato `FST_ERR_...`.

3.4 Integração com TypeScript

Frameworks anteriores ao TypeScript costumam oferecer tipagem por meio de declarações externas, que descrevem a biblioteca sem participar dela. O Fastify adota outro caminho, com os **type providers**: um adaptador ensina ao compilador como derivar tipos a partir dos schemas declarados na rota.

O efeito prático aparece no autocompletar e, principalmente, nos erros de compilação. Acessar um campo que não existe no schema, ou devolver um objeto que não corresponde ao formato declarado na resposta, deixa de ser um problema descoberto em produção e passa a ser um erro de compilação.

4 Mapeamento objeto-relacional (ORM)

4.1 O descompasso objeto-relacional

Bancos relacionais e linguagens orientadas a objetos organizam informação de maneiras diferentes. O modelo relacional trabalha com conjuntos de tuplas, identidade determinada por valor de chave e relacionamentos expressos por referência entre tabelas. Linguagens de aplicação trabalham com objetos, identidade por referência em memória, herança e listas aninhadas.

Essa diferença estrutural é conhecida como **descompasso de impedância objeto-relacional**. Ela se manifesta em perguntas cotidianas: quando um objeto `Categoria` é carregado, seus `itens` vêm junto? Se vierem sempre, listar categorias pode arrastar a tabela inteira de produtos. Se nunca vierem, cada acesso dispara uma consulta nova. Não existe resposta universal — é justamente por isso que o problema é difícil.

4.2 Active Record e Data Mapper

A literatura de padrões de arquitetura descreve duas respostas clássicas. No **Active Record**, o objeto que representa a linha também sabe persisti-la: `categoria.save()`. O modelo é direto e produtivo, mas mistura dados e acesso ao banco na mesma classe.

No **Data Mapper**, uma camada separada traduz entre objetos e tabelas. Os objetos de domínio não sabem que existe um banco, o que favorece testes e separação de responsabilidades, ao custo de mais estrutura.

O Drizzle não se encaixa exatamente em nenhum dos dois. Ele não produz objetos com métodos de persistência nem mantém um mapa de identidade: as consultas devolvem **objetos simples**, e as escritas são expressas como operações explícitas. Essa posição o aproxima de um query builder fortemente tipado, com um mapeamento de relações por cima.

4.3 ORM, query builder e SQL puro

Abordagem	Controle sobre o SQL	Segurança de tipos	Custo principal
SQL puro (driver)	Total	Nenhuma por padrão	Verbosidade e mapeamento manual
Query builder	Alto	Parcial a alta	API própria a aprender
ORM completo	Baixo a médio	Alta	Comportamento implícito

A escolha não é sobre qual abordagem é superior, e sim sobre qual risco se prefere assumir. SQL puro oferece controle total e nenhuma rede de proteção contra erros de digitação em nomes de coluna. ORMs completos oferecem a maior proteção e o menor controle: uma consulta ineficiente pode ser gerada sem que isso fique visível no código.

4.4 O problema N+1

O efeito colateral mais conhecido de ORMs é o **problema N+1**. Ele ocorre quando uma consulta devolve N registros e o código, ao iterar sobre eles, dispara uma consulta adicional por registro para carregar dados relacionados. Listar 50 categorias e, dentro do laço, acessar os itens de cada uma resulta em 51 consultas.

O problema raramente aparece em desenvolvimento, onde as tabelas são pequenas e o banco é local. Ele se manifesta em produção, quando o volume cresce e a latência de rede multiplica o custo de cada ida ao banco. Evitá-lo exige que a ferramenta ofereça uma forma explícita de carregar relações junto — e que o programador saiba que precisa usá-la.

5 Drizzle ORM: particularidades

5.1 O schema em TypeScript como fonte única

No Drizzle, a estrutura do banco é declarada em TypeScript. Não há linguagem própria de modelagem nem etapa de geração de código: o schema é código comum, que o editor entende e o compilador verifica.

```
export const categorias = pgTable(
  'categorias',
  {
    id: uuid('id').primaryKey().defaultRandom(),
    empresaId: uuid('empresa_id')
      .notNull()
      .references(() => empresas.id, { onDelete: 'cascade' }),
    nome: varchar('nome', { length: 100 }).notNull(),
  },
  (table) => [
    index('categorias_empresa_id_idx').on(table.empresaId),
    unique('categorias_empresa_id_nome_unique').on(table.empresaId, table.nome),
  ],
)

export type Categoria = typeof categorias.$inferSelect
export type NovaCategoria = typeof categorias.$inferInsert
```

Declaração da tabela `categorias`. Os tipos `Categoria` e `NovaCategoria` são derivados da própria definição — alterar uma coluna atualiza os tipos automaticamente.

As duas últimas linhas merecem atenção. `$inferSelect` e `$inferInsert` derivam, da definição da tabela,

o tipo de uma linha lida e o de uma linha a inserir — que são diferentes, já que colunas com valor padrão são opcionais na inserção. Como são **derivados**, não há como o tipo divergir da tabela: renomear uma coluna quebra a compilação onde ela era usada.

5.2 Duas APIs de consulta

O Drizzle oferece dois estilos de consulta, e saber quando usar cada um é parte de usar a ferramenta bem.

O primeiro é a **API no estilo SQL**, em que os métodos correspondem a cláusulas: `select`, `from`, `where`, `innerJoin`, `orderBy`. A consulta escrita se parece com o SQL que será executado, o que torna o custo de cada operação visível.

O segundo é a **Relational Query API**, acessível por `db.query`, que carrega relações declaradas por meio da opção `with` e devolve objetos aninhados. É mais conveniente para leituras que precisam montar um agregado, e resolve o problema N+1 em uma única ida ao banco.

Neste projeto, as duas aparecem conforme a necessidade. A leitura de um prato com sua composição usa `db.query`; já a listagem paginada dos itens de um prato usa `innerJoin` explícito, porque a paginação precisa incidir sobre a **relação**, e não sobre o registro raiz — algo que a Relational Query API não expressa.

5.3 Migrations com drizzle-kit

O `drizzle-kit` compara o schema declarado com o estado já registrado e gera o SQL da diferença. O resultado é um arquivo `.sql` legível, versionado junto ao código — e não um artefato opaco.

Essa transparência importa porque migrations são um dos pontos em que abstrações falham de forma cara. Renomear uma coluna é, para um comparador automático, indistinguível de apagar uma coluna e criar outra. Poder ler e corrigir o SQL antes de aplicá-lo é o que evita perda de dados.

```
ALTER TABLE "itens" ADD CONSTRAINT "itens_categoria_id_categorias_id_fk"
FOREIGN KEY ("categoria_id") REFERENCES "public"."categorias"("id")
ON DELETE restrict ON UPDATE cascade;
```

Trecho da migration gerada para este projeto. O nome da constraint segue um padrão previsível — fato aproveitado para traduzir erros do banco em mensagens de usuário.

5.4 Comparação com Prisma e TypeORM

Aspecto	Drizzle	Prisma	TypeORM
Definição do schema	TypeScript	Arquivo .prisma	Decorators/classes
Geração de código	Não exige	Exige (generate)	Não exige
Padrão arquitetural	Query builder tipado	Client gerado	Active Record / Data Mapper
Proximidade do SQL	Alta	Baixa	Média

A diferença mais relevante para quem está aprendendo é a **proximidade do SQL**. Ferramentas que afastam o programador do SQL aceleram o início e dificultam o diagnóstico depois: quando uma consulta fica lenta, é preciso descobrir o que foi gerado e por quê. O Drizzle faz a aposta oposta — exige conhecer SQL desde o começo e, em troca, mantém previsível o que será executado.

6 Relacionamento entre produto e categoria com Drizzle

Esta seção trata do ponto central da atividade. O caso de estudo é o relacionamento entre `itens` (produto) e `categorias`: **uma categoria agrupa vários itens, e cada item pertence a exatamente uma categoria** — um relacionamento um-para-muitos.

6.1 Os dois lados da implementação

Implementar esse relacionamento no Drizzle envolve duas declarações que são frequentemente confundidas, mas que operam em camadas distintas e são independentes uma da outra:

- `.references()` — declara a **chave estrangeira no banco de dados**. Vira DDL na migration e passa a ser aplicada pelo PostgreSQL, independentemente da aplicação que estiver acessando os dados.
- `relations()` — declara o **grafo de relações para a aplicação**. Não gera nada no banco: é o que habilita a Relational Query API e a tipagem do resultado aninhado.

Esta é a confusão mais comum ao aprender Drizzle. Declarar apenas `relations()` produz uma API que *parece* funcionar — as consultas aninhadas respondem — mas sem nenhuma garantia de integridade: o banco aceitaria um item apontando para uma categoria inexistente. Declarar apenas `.references()` garante a integridade, mas `db.query` não consegue navegar pela relação. As duas são necessárias, e por motivos diferentes.

6.2 A chave estrangeira no banco

A coluna `categoria_id` é declarada na tabela `itens` com `.references()`, apontando para a chave primária de `categorias`:

```
export const itens = pgTable('itens', {
  id: uuid('id').primaryKey().defaultRandom(),
  empresaId: uuid('empresa_id')
    .notNull()
    .references(() => empresas.id, { onDelete: 'cascade' }),
  categoriaId: uuid('categoria_id')
    .notNull()
    .references(() => categorias.id, { onDelete: 'restrict' }),
  nome: varchar('nome', { length: 150 }).notNull(),
  preco: numeric('preco', { precision: 10, scale: 2, mode: 'number' })
    .notNull()
    .default(0),
  ativo: boolean('ativo').notNull().default(true),
}, (table) => [
  index('itens_categoria_id_idx').on(table.categoriaId),
])
```

A tabela `itens` com as duas chaves estrangeiras do modelo.

Três detalhes dessa declaração merecem comentário. O primeiro é que a referência é escrita como **função** — `() => categorias.id` — e não como valor direto. Isso adia a resolução da dependência para depois que ambos os módulos foram carregados, permitindo que duas tabelas se referenciem mutuamente sem quebrar por ordem de importação.

O segundo é o `index` sobre `categoria_id`. O PostgreSQL cria índice automaticamente para chaves primárias e constraints de unicidade, mas **não** para chaves estrangeiras. Sem ele, toda consulta que

filtra itens por categoria percorre a tabela inteira.

O terceiro é o `mode: 'number'` em `preco`. O tipo `numeric` do PostgreSQL tem precisão arbitrária, maior do que a do `number` do JavaScript; por isso o driver o devolve como string por padrão. Para valores monetários na faixa usual, converter para número é seguro e torna a resposta da API mais natural — mas é uma escolha consciente, não um detalhe.

6.3 O grafo de relações na aplicação

Com as chaves estrangeiras declaradas, o segundo passo é descrever as relações para a aplicação. Neste projeto elas ficam em um arquivo próprio, `src/db/schema/relations.ts`, o que evita importações circulares entre os arquivos de tabela e separa claramente o que descreve o banco do que descreve o modelo de leitura:

```
// Lado 1: uma categoria tem muitos itens
export const categoriasRelations = relations(categorias, ({ one, many }) => ({
  empresa: one(empresas, {
    fields: [categorias.empresaId],
    references: [empresas.id],
  }),
  itens: many(itens),
}))

// Lado N: um item pertence a uma categoria
export const itensRelations = relations(itens, ({ one, many }) => ({
  empresa: one(empresas, {
    fields: [itens.empresaId],
    references: [empresas.id],
  }),
  categoria: one(categorias, {
    fields: [itens.categoriaId],
    references: [categorias.id],
  }),
  pratoItens: many(pratoItens),
}))
```

Os dois lados do relacionamento um-para-muitos entre categoria e item.

A assimetria entre os dois lados não é arbitrária. O lado `many` não precisa de `fields` e `references` porque a informação de ligação está na outra tabela: é o item que guarda `categoria_id`. Já o lado `one` precisa indicar explicitamente qual coluna liga a qual, e isso se torna essencial quando há **mais de uma relação para a mesma tabela**. A tabela `itens` aponta duas vezes para tabelas distintas (`empresas` e `categorias`); se apontasse duas vezes para a mesma, apenas os `fields` permitiriam distinguir as relações.

É importante notar que `relations()` não sabe nada sobre `.references()`. Nada impede declarar uma relação entre colunas que não tenham chave estrangeira — a consulta funcionaria, e a integridade simplesmente não existiria. São duas declarações paralelas sobre o mesmo par de colunas, e mantê-las coerentes é responsabilidade de quem escreve o schema.

6.4 Consultando os dois lados

Declaradas as relações, a Relational Query API permite navegar em qualquer direção. Do lado da categoria, carregando seus itens:

```
const categoria = await db.query.categorias.findFirst({
  where: eq(categorias.id, id),
  columns: { id: true, empresaId: true, nome: true },
  with: {
    empresa: { columns: { id: true, nome: true } },
    itens: {
      columns: { id: true, nome: true, preco: true, ativo: true },
      orderBy: asc(itens.nome),
    },
  },
})
```

Leitura de uma categoria com a empresa dona e a lista de itens.

E do lado do item, carregando a categoria a que pertence:

```
const item = await db.query.itens.findFirst({
  where: eq(itens.id, id),
  with: {
    empresa: { columns: { id: true, nome: true } },
    categoria: { columns: { id: true, nome: true } },
  },
})
```

Leitura de um item com suas duas relações [one](#) resolvidas.

Em ambos os casos, o retorno é integralmente tipado: o editor sabe que `categoria.itens` é um array de objetos com exatamente os quatro campos pedidos, e acessar qualquer outro campo é erro de compilação. A opção `columns` também cumpre um papel de desempenho — apenas as colunas declaradas entram no `SELECT`.

6.5 O SQL efetivamente gerado

A afirmação de que a Relational Query API resolve o N+1 em uma única consulta pode ser verificada. Chamando `.toSQL()` sobre a consulta de categorias com seus itens, o SQL produzido para PostgreSQL é o seguinte (reformatado para leitura):

```
select "categorias"."id", "categorias"."empresa_id", "categorias"."nome",
       "categorias_itens"."data" as "itens"
from "categorias" "categorias"
left join lateral (
  select coalesce(
    json_agg(json_build_array(
      "categorias_itens"."id",
      "categorias_itens"."empresa_id",
      "categorias_itens"."categoria_id",
      "categorias_itens"."nome",
      "categorias_itens"."preco",
      "categorias_itens"."ativo"
    )), '[]'::json
  ) as "data"
from "itens" "categorias_itens"
where "categorias_itens"."categoria_id" = "categorias"."id"
) "categorias_itens" on true
```

SQL gerado pela Relational Query API do Drizzle para o relacionamento 1:N.

A estrutura revela a estratégia adotada. Em vez de emitir uma consulta por categoria, ou de trazer o produto cartesiano de um `JOIN` comum — que repetiria os dados da categoria uma vez por item —, o Drizzle usa um **LEFT JOIN LATERAL** com agregação em JSON. A subconsulta lateral pode

referenciar a linha externa (`categorias.id`), e `json_agg` colapsa os itens em um único valor por categoria.

Três consequências práticas decorrem dessa escolha. Os dados da categoria trafegam uma única vez, independentemente do número de itens. O `coalesce(..., '[]'::json)` garante que uma categoria sem itens devolva um array vazio, e não `null` — o que dispensa verificações no código da aplicação. E, sobretudo, **é uma consulta só**, o que preserva o custo constante em idas ao banco mesmo quando o número de categorias cresce.

6.6 Extensão para muitos-para-muitos

O modelo desta atividade inclui também um relacionamento muitos-para-muitos: um prato é composto por vários itens, e um mesmo item participa de vários pratos. Como o modelo relacional não representa essa cardinalidade diretamente, ela é decomposta em duas relações um-para-muitos por meio de uma **tabela de junção**, `prato_itens`.

```
export const pratoItens = pgTable('prato_itens', {
  id: uuid('id').primaryKey().defaultRandom(),
  pratoId: uuid('prato_id').notNull()
    .references(() => pratos.id, { onDelete: 'cascade' }),
  itemId: uuid('item_id').notNull()
    .references(() => itens.id, { onDelete: 'cascade' }),
}, (table) => [
  // Impede que o mesmo item seja vinculado duas vezes ao mesmo prato
  unique('prato_itens_prato_id_item_id_unique').on(table.pratoId, table.itemId),
])

export const pratoItensRelations = relations(pratoItens, ({ one }) => ({
  prato: one(pratos, { fields: [pratoItens.pratoId], references: [pratos.id] }),
  item: one(itens, { fields: [pratoItens.itemId], references: [itens.id] }),
})))
```

Tabela de junção e suas duas relações `one` — uma para cada ponta do N:N.

A constraint de unicidade sobre o par (`prato_id`, `item_id`) merece destaque. Sem ela, nada impediria vincular o mesmo item duas vezes ao mesmo prato, e a regra teria de ser verificada na aplicação — com uma consulta prévia sujeita a condição de corrida entre a verificação e a inserção. Declarada no banco, a garantia é absoluta e custa uma tentativa de inserção que falha.

Na Relational Query API do Drizzle, a tabela de junção **não é escondida**: a navegação é feita em duas etapas, `prato` para `pratoItens` e `pratoItens` para `item`. O achatamento, quando desejado, fica explícito no código da aplicação:

```
const prato = await db.query.pratos.findFirst({
  where: eq(pratos.id, id),
  with: {
    pratoItens: {
      columns: {}, // descarta as colunas da tabela de juncao
      with: {
        item: {
          columns: { id: true, nome: true, preco: true, ativo: true },
          with: { categoria: { columns: { id: true, nome: true } } },
        },
      },
    },
  },
})

// prato -> pratoItens -> item -> categoria, tudo em uma consulta
const { pratoItens: vinculos, ...resto } = prato
const itensDoPrato = vinculos.map((vinculo) => vinculo.item)
```

Travessia do N:N com três níveis de aninhamento, resolvida em uma única ida ao banco.

O `columns: {}` é o detalhe que torna o resultado limpo: ele instrui o Drizzle a não trazer nenhuma coluna da tabela de junção, que interessa apenas como caminho até o item. Exigir essa etapa explícita é uma escolha de projeto coerente com a filosofia da ferramenta — a tabela de junção existe no banco, e o código a reflete.

6.7 Integridade referencial: restrict e cascade

Declarar a chave estrangeira levanta uma pergunta que não pode ficar sem resposta: o que acontece quando o registro referenciado é removido? O PostgreSQL oferece várias políticas, e neste projeto foram usadas duas, escolhidas caso a caso.

Relação	ON DELETE	Justificativa
itens -> categorias	<code>restrict</code>	Apagar deixaria o item sem classificação
itens -> empresas	<code>cascade</code>	O item não existe fora da empresa
categorias -> empresas	<code>cascade</code>	Idem
prato_itens -> pratos	<code>cascade</code>	O vínculo não existe sem as duas pontas
cidades -> ufs	<code>restrict</code>	Perda silenciosa de dados

O critério adotado foi de **propriedade**: quando o registro filho só faz sentido dentro do pai, usa-se `cascade`; quando ele tem existência própria e apenas se apoia no outro, usa-se `restrict`, que transforma a remoção em erro explícito em vez de perda silenciosa.

Um detalhe prático surgiu ao traduzir esses erros para a API. As duas situações produzem códigos SQLSTATE **diferentes**: tentar apagar uma categoria que ainda tem itens gera `23001 (restrict_violation)`, enquanto inserir um item apontando para categoria inexistente gera `23503 (foreign_key_violation)`. São problemas opostos — um é remoção bloqueada, o outro é referência inválida — e distingui-los permite mensagens precisas:

```
// DELETE barrado porque ainda existem registros dependentes
if (erroDoBanco.code === PG_ERROR.RESTRICT_VIOLATION) {
  return reply.status(409).send({
    message: RESTRICT_MESSAGES[constraint]
    ?? 'Nao e possivel excluir: existem registros vinculados a este.',
    code: 'RESTRICT_VIOLATION',
  })
}

// Referencia apontando para um registro que nao existe
if (erroDoBanco.code === PG_ERROR.FOREIGN_KEY_VIOLATION) {
  return reply.status(409).send({
    message: CONSTRAINT_MESSAGES[constraint]
    ?? 'Um dos registros relacionados nao existe.',
    code: 'FOREIGN_KEY_VIOLATION',
  })
}
```

Distinção entre remoção bloqueada e referência inválida no tratador central.

6.8 O que a chave estrangeira não garante

A última observação desta seção diz respeito aos limites da integridade referencial. A tabela `itens` possui duas chaves estrangeiras independentes: `empresa_id` e `categoria_id`. Cada uma garante que o registro apontado existe — mas nenhuma das duas, nem as duas juntas, garante que sejam **coerentes entre si**.

Concretamente: nada no banco impede gravar um item da empresa A classificado em uma categoria da empresa B. Ambas as chaves estrangeiras estão satisfeitas, e o estado resultante é inválido do ponto de vista do negócio. Garanti-lo no banco exigiria uma chave estrangeira composta, o que implicaria alterar as chaves primárias do modelo dado.

A alternativa adotada foi verificar a regra explicitamente na aplicação, antes de gravar:

```
async function garantirCategoriaDaEmpresa(categoriaId: string, empresaId: string) {
  const categoria = await db.query.categorias.findFirst({
    where: eq(categorias.id, categoriaId),
    columns: { id: true, empresaId: true },
  })

  if (!categoria) throw new NotFoundError('Categoria', categoriaId)

  if (categoria.empresaId !== empresaId) {
    throw new ConflictError(
      'A categoria informada pertence a outra empresa. ' +
      'Um item so pode usar categorias da propria empresa.',
    )
  }
}
```

Regra de coerência entre as duas chaves estrangeiras, verificada na aplicação.

O caso é instrutivo porque delimita até onde o modelo relacional protege sozinho. Chaves estrangeiras garantem que referências apontem para registros existentes; não garantem que o conjunto de referências de uma linha faça sentido. Reconhecer essa fronteira é o que evita dois erros simétricos: confiar que o banco valida tudo, ou desconfiar dele a ponto de replicar na aplicação verificações que ele já faz melhor.

7 Considerações finais

O percurso deste artigo sugere que frameworks e ORMs são mais bem avaliados pelo que tornam **visível** do que pelo que conseguem esconder. O Fastify não se destaca por eliminar a validação, e sim por exigir que o contrato de cada rota seja declarado — e, tendo-o, reaproveitá-lo na tipagem, na serialização e na documentação. O Drizzle não se destaca por dispensar o conhecimento de SQL, e sim por manter previsível o SQL que será executado.

O relacionamento entre produto e categoria ilustra bem esse ponto. Sua implementação exigiu duas declarações que à primeira vista parecem redundantes, mas que operam em camadas distintas: `.references()`, que cria a restrição no banco, e `relations()`, que descreve o grafo para a aplicação. Compreender por que ambas são necessárias, e o que exatamente cada uma garante, é o que separa usar a ferramenta de entendê-la.

Vale registrar, por fim, uma lição de método. Duas das decisões discutidas aqui — o desempacotamento do erro do driver de dentro do `DrizzleQueryError` e a distinção entre os códigos 23001 e 23503 — não vieram da documentação, mas de executar a API contra um banco real e observar o que acontecia. Ambas passariam despercebidas em uma verificação apenas de compilação: o código compilava, os tipos estavam corretos, e o comportamento estava errado. Tipos não substituem execução.

Referências

FASTIFY. **Fastify Documentation**. Disponível em: <https://fastify.dev/docs/latest/>. Acesso em: setembro de 2026.

FASTIFY. **Validation and Serialization**. Disponível em: <https://fastify.dev/docs/latest/Reference/Validation-and-Serialization/>. Acesso em: setembro de 2026.

FASTIFY. **Plugins and Encapsulation**. Disponível em: <https://fastify.dev/docs/latest/Reference/Plugins/>. Acesso em: setembro de 2026.

DRIZZLE TEAM. **Drizzle ORM Documentation**. Disponível em: <https://orm.drizzle.team/docs/overview>. Acesso em: setembro de 2026.

DRIZZLE TEAM. **Drizzle Relations**. Disponível em: <https://orm.drizzle.team/docs/relations>. Acesso em: setembro de 2026.

DRIZZLE TEAM. **Drizzle Kit: migrations**. Disponível em: <https://orm.drizzle.team/docs/kit-overview>. Acesso em: setembro de 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation: Appendix A — Error Codes**. Disponível em: <https://www.postgresql.org/docs/current/errcodes-appendix.html>. Acesso em: setembro de 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation: CREATE TABLE — REFERENCES**. Disponível em: <https://www.postgresql.org/docs/current/sql-createtable.html>. Acesso em: setembro de 2026.

FOWLER, Martin. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2002.

FOWLER, Martin. **InversionOfControl**. Disponível em:

<https://martinfowler.com/bliki/InversionOfControl.html>. Acesso em: setembro de 2026.

ZOD. **Zod Documentation**. Disponível em: <https://zod.dev>. Acesso em: setembro de 2026.

NODE.JS. **Node.js Documentation**. Disponível em: <https://nodejs.org/docs/latest/api/>. Acesso em: setembro de 2026.